

Docker Lab

In this lab, we are going to drill into Docker.

What is Virtualization?

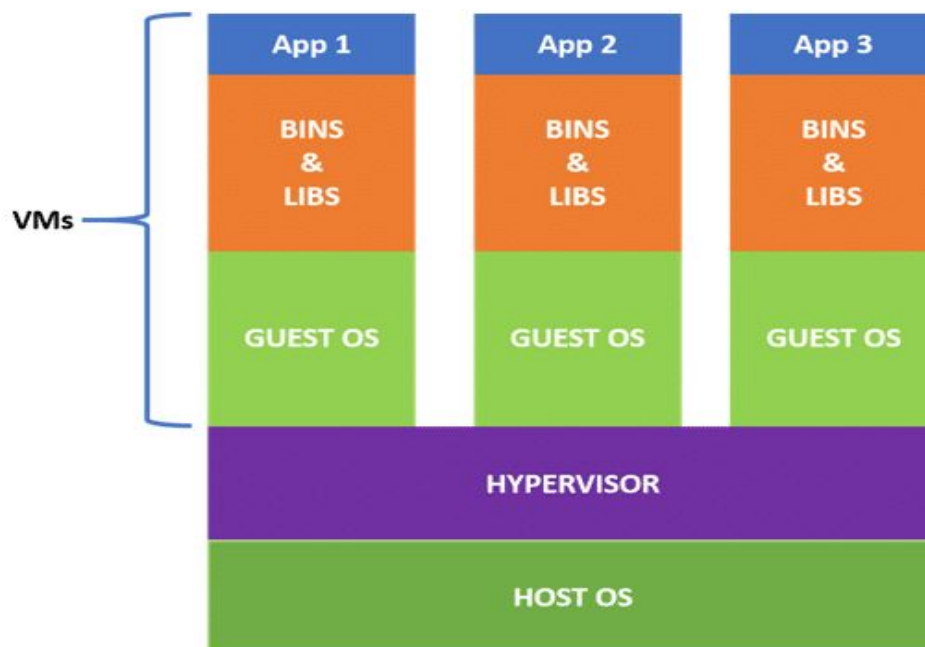
Virtualization refers to importing a guest operating system on our host operating system, allowing developers to run multiple OS on different VMs while all of them run on the same host, thereby eliminating the need to provide extra hardware resources.

Advantages of Virtualization:

VMs are being used in the industry in the following ways:

- Enable multiple OS on the same machine
- Cheaper due to less/compact infrastructure setup
- Easy to recover in case of failure
- Easy for maintenance
- Faster provisioning of applications and resources required for tasks
- Increase in IT productivity

The following is the architecture for virtualization:



What is a virtualization host?

In above, the three guest operating systems acting as virtual machines are running on a host operating system. The process of manually reconfiguring hardware and firmware and installing a new OS can be entirely automated; all these steps get stored as data in any files of a disk.

Virtualization lets us run our applications on fewer physical servers. In virtualization, each application and operating system live in a separate software container called VM. Where VMs are completely isolated, all the computing resources like CPUs, storage, and networking are pooled together, and they are delivered dynamically to each VM by a software called a hypervisor.

However, running multiple VMs over the same host leads to degradation in performance. As guest operating systems have their own kernel, libraries, and many dependencies running on a single host OS, it takes up a large occupation of resources such as the processor, hard disk and, especially, its RAM. Also, when we use VMs in virtualization, the bootup process takes a long time that would affect efficiency in the case of real-time applications.

Disadvantages of Virtualization:

- Running multiple VMs leads to unstable performance
- Hypervisors are not as efficient as the host operating system
- Boot up process is long and takes time

These drawbacks led to the emergence of a new technique called Containerization.

What is Containerization?

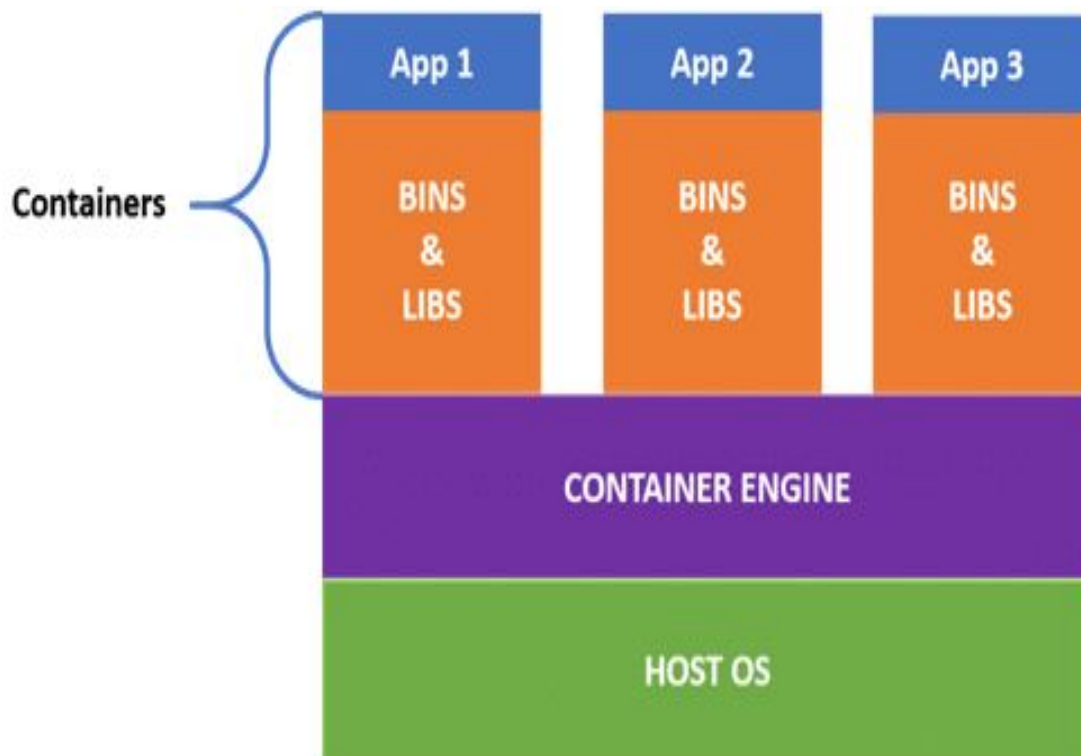
Containerization is a technique where the virtualization is brought to an operating system level. **In containerization, we virtualize the operating system resources.** It is more efficient as there is **no guest operating system consuming the host resources;** instead, **containers utilize only the host operating system** and share relevant libraries

and resources only when they are required. The required binaries and libraries of containers run on the host kernel leading to faster processing and execution.

In a nutshell, containerization (containers) is a lightweight virtualization technology acting as an alternative to hypervisor virtualization. Any application can be bundled in a container and run without caring about dependencies, libraries, and binaries.

In the case of containerization, **all containers share the same host operating system**. Multiple containers get created for every type of application making them faster but without wasting the resources, unlike virtualization where a kernel is required for every OS and lots of resources from the host OS are utilized.

The diagram below makes it clear:



Advantages of Containers:

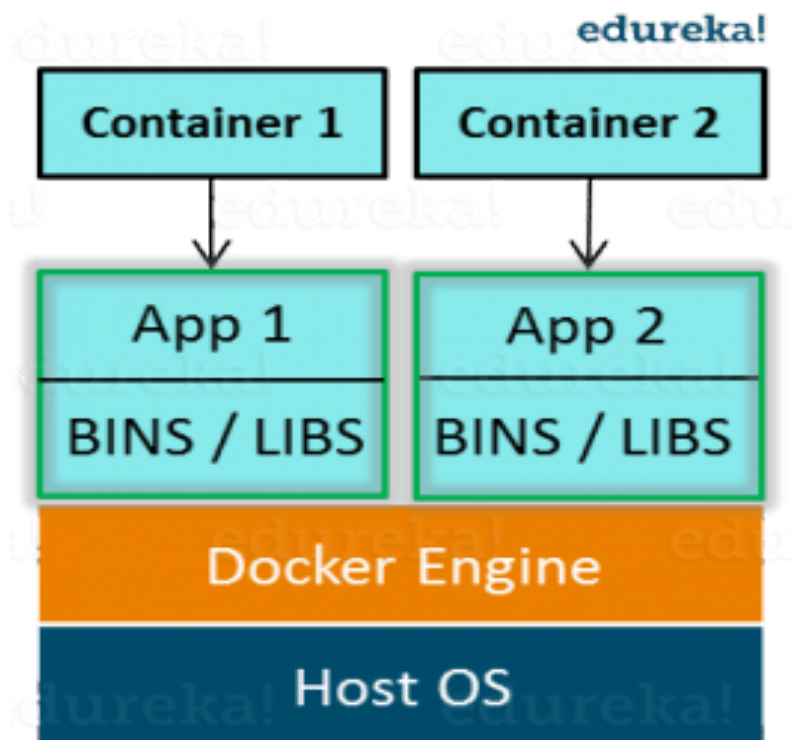
- Containers are small and lightweight as they share the same OS kernel.
- They do not take much time to boot-up (only seconds).
- They exhibit high performance with lower resource utilization.

Containerization vs Virtualization

Containerization	Virtualization
Virtualizes OS	Virtualizes hardware
Installs container only on host OS	Requires complete OS install per VM
Uses only kernel of host OS	Installs a separate kernel per VM
Lightweight	Heavyweight
Native performance	Limited performance
Process level isolation	Fully isolated

Docker

Docker is a containerization platform that packages your application and all its dependencies together in the form of Containers to ensure that your application works seamlessly in any environment.



Each application will run on a separate container and will have its own set of libraries and dependencies. This also ensures that there is process level isolation, meaning each application is independent of other applications, giving developers surety that they can build applications that will not interfere with one another.

As a developer, one can build a container which has different applications installed on it and give it to my QA team who will only need to run the container to replicate the developer environment.

Benefits of Docker:

With docker, the QA team need not install all the dependent software and applications to test the code and this helps them save lots of time and energy. This also ensures that the working environment is consistent across all the individuals involved in the process, starting from development to deployment. The number of systems can be scaled up easily and the code can be deployed on them effortlessly.

Docker Architecture

Docker uses a client–server architecture. The Docker client consists of Docker build, Docker pull, and Docker run. The client approaches the Docker daemon that further helps in building, running, and distributing Docker containers. Docker client and Docker daemon can be operated on the same system; otherwise, we can connect the Docker client to the remote Docker daemon. Both communicate with each other using the REST API, over UNIX sockets or a network.

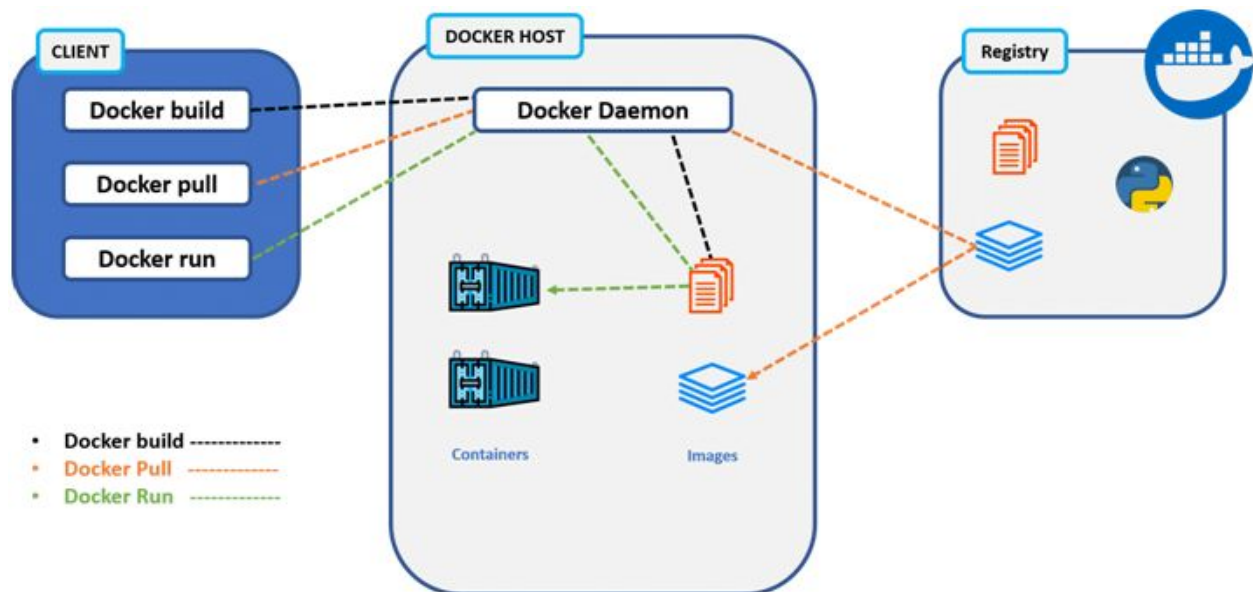
The basic architecture in Docker consists of three parts:

- Docker Client
- Docker Host
- Docker Registry

Docker Client

- It is the primary way for many Docker users to interact with Docker.
- It uses command-line utility or other tools that use Docker API to communicate with the Docker daemon.

- A Docker client can communicate with more than one Docker daemon.



Docker Host

In Docker host, we have Docker daemon and Docker objects such as containers and images. First, let's understand the objects on the Docker host, then we will proceed toward the functioning of the Docker daemon.

- **Docker Image:** A Docker image is a Docker object. It is a type of recipe/template that can be used for creating Docker containers. It includes steps for creating the necessary software.
- **Docker Container:** A Docker container is also a Docker object. A type of virtual machine created from the instructions found within the Docker image. It is a running instance of a Docker image that consists of the entire package required to run an application.
- **Docker Daemon:**
 - Docker daemon helps in listening requests for the Docker API and in managing Docker objects such as images, containers, volumes, etc. Daemon issues to build an image based on a user's input and then saves it in the registry.
 - In case we don't want to create an image, then we can simply pull an image from the Docker hub (which might be built by some other user). In case we want to create a running instance of our Docker image, then we need to issue a run command that would create a Docker container.

- A Docker daemon can communicate with other daemons to manage Docker services.

Docker Registry and Docker Hub:

- Docker registry is a repository for Docker images which is used for creating Docker containers.
- We can use a local/private registry or the Docker hub, which is the most popular social example of a Docker repository.

Docker Hub is like GitHub for Docker Images. It is basically a cloud registry where you can find Docker Images uploaded by different communities, also you can develop your own image and upload on Docker Hub, but first, you need to create an account on DockerHub.

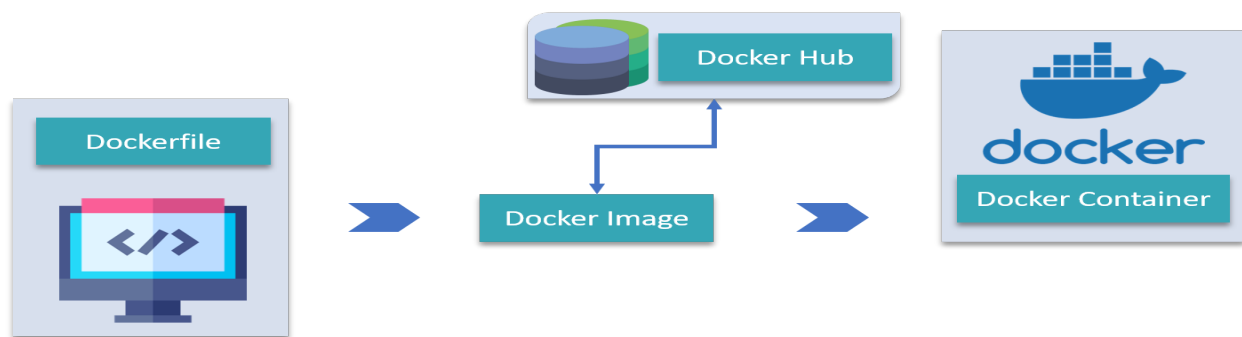
Dockerfile:

- You specify what to include in your Docker container via a special file which by convention is called *Dockerfile*.
- The Dockerfile contains a set of Docker instructions which are executed by the Docker command line tool. The result is a Docker image

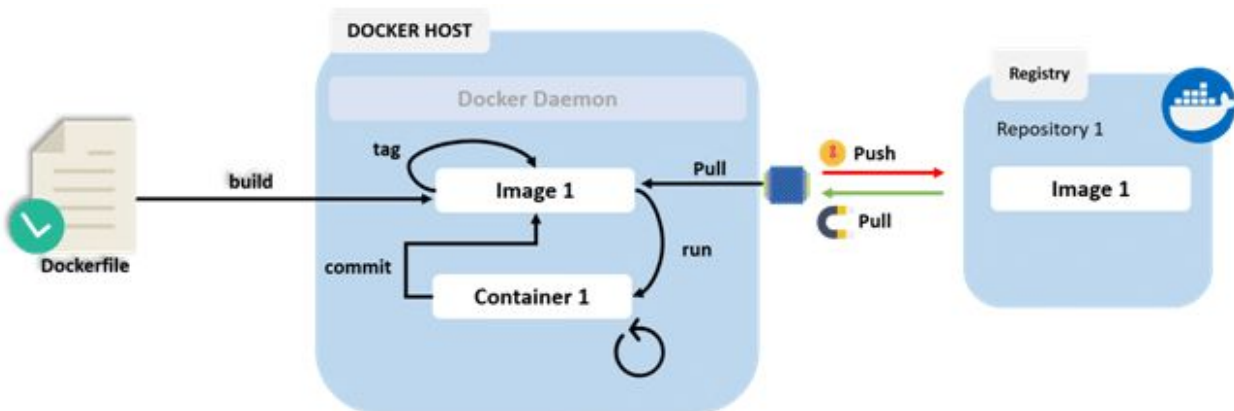
Specifically,

1. Docker Image is created by the sequence of commands written in the Dockerfile.
2. When this Dockerfile is executed using a docker command it results into a Docker Image with a name.
3. When this Image is executed by “docker run” command it will by itself start whatever application or service it must start on its execution.

The following diagram clears this concept:



The following is a more detailed workflow:



- It basically involves building an image from a Dockerfile that consists of instructions about container configuration or image pulling from a Docker registry, like Docker hub.
- When this image is built in our Docker environment, we should be able to run the image which further creates a container.
- In our container, we can do any operations such as:
 - Stopping the container
 - Starting the container
 - Restarting the container
- These runnable containers can be started, stopped, or restarted just like how we operate a virtual machine or a computer.
- Whatever manual changes are made such as configurations or software installations, these changes in a container can be committed to making a new image, which can further be used for creating a container from it later.
- At last, when we want to share our image with our team or to the world, we can easily push our image into a Docker registry.
- One can easily pull this image from the Docker registry using the pull command.

Docker Compose

- The Docker Compose feature enables you to "link" multiple Docker containers into a single "composition", which can be installed / deployed and started up all at once.
- Docker Compose is basically used to run multiple Docker Containers as a single server. Suppose if I have an application which requires WordPress, Maria DB

and PHP MyAdmin. I can create one file which would start both the containers as a service without the need to start each one separately. It is really useful especially if you have a microservice architecture.

- For instance, an application in one Docker container, and a database in another Docker container - in case both Docker containers are necessary for the application to run.

Docker Installation

- Access docs.docker.com
- Click on Get Docker
- Select Docker for Linux
- Select Ubuntu from menu

The screenshot shows the Docker documentation website. The browser address bar displays `docs.docker.com/engine/install/ubuntu`. The page title is "Install Docker Engine on Ubuntu" with an estimated reading time of 11 minutes. The left sidebar contains a navigation menu with categories like "Docker Engine", "Docker Compose", "Docker Desktop", "Docker Hub", "Open-source projects", "Release notes", and "Superseded products and tools". The "Docker Engine" section is expanded, showing options like "Overview", "Install", "Installation per distro", "Release notes", and "Previous versions". The "Installation per distro" section is further expanded, listing "Install on CentOS", "Install on Debian", "Install on Fedora", "Install on Ubuntu" (which is selected), "Install binaries", and "Optional post-installation steps". The main content area for "Install Docker Engine on Ubuntu" includes a "Prerequisites" section with "OS requirements" (listing Ubuntu versions: Eoan 19.10, Bionic 18.04 (LTS), Xenial 16.04 (LTS)) and "Uninstall old versions" (providing a terminal command: `$ sudo apt-get remove docker docker-engine docker.io containerd runc`). It also mentions "Supported storage drivers" (listing `overlay2`, `aufs`, and `btrfs`). The right sidebar contains a "On this page" section with links to various parts of the page, including "Prerequisites", "OS requirements", "Uninstall old versions", "Supported storage drivers", "Installation methods", "Install using the repository", "Set up the repository", "Install Docker Engine", "Upgrade Docker Engine", "Install from a package", "Upgrade Docker Engine", "Install using the convenience script", "Upgrade Docker after using the convenience script", "Uninstall Docker Engine", and "Next steps".

To install Docker Engine, you need the 64-bit version of one of these Ubuntu versions:

- Ubuntu Eoan 19.10
- Ubuntu Bionic 18.04 (LTS)
- Ubuntu Xenial 16.04 (LTS)

`cat /etc/*release*`

to cross check dependencies

```
tariqjc@tariqjc:~$ cat /etc/*release*
DISTRIB_ID=Ubuntu
DISTRIB_RELEASE=18.04
DISTRIB_CODENAME=bionic
DISTRIB_DESCRIPTION="Ubuntu 18.04.3 LTS"
NAME="Ubuntu"
VERSION="18.04.3 LTS (Bionic Beaver)"
ID=ubuntu
ID_LIKE=debian
PRETTY_NAME="Ubuntu 18.04.3 LTS"
VERSION_ID="18.04"
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
VERSION_CODENAME=bionic
UBUNTU_CODENAME=bionic
```

uninstall older versions: Older versions of Docker were called docker, docker.io, or docker-engine. Uninstall them. New version is docker-ce

`sudo apt-get remove docker docker-engine docker.io containerd runc`

```
tariqjc@tariqjc: ~
File Edit View Search Terminal Help
tariqjc@tariqjc:~$ sudo apt-get remove docker docker-engine docker.io containerd
runc
[sudo] password for tariqjc:
Reading package lists... Done
Building dependency tree
Reading state information... Done
Package 'docker-engine' is not installed, so not removed
Package 'docker' is not installed, so not removed
Package 'containerd' is not installed, so not removed
Package 'docker.io' is not installed, so not removed
Package 'runc' is not installed, so not removed
0 upgraded, 0 newly installed, 0 to remove and 135 not upgraded.
```

To uninstall previously installed docker-ce, use:

```
sudo apt-get purge docker-ce docker-ce-cli containerd.io
```

Images, containers, volumes, or customized configuration files on your host are not automatically removed. To delete all images, containers, and volumes:

```
sudo rm -rf /var/lib/docker
```

Execute the following commands to setup the repository

```
tariqjc@tariqjc:~$ sudo apt-get update
Ign:1 http://dl.google.com/linux/chrome/deb stable InRelease
Hit:2 http://security.ubuntu.com/ubuntu bionic-security InRelease
Hit:3 https://download.docker.com/linux/ubuntu bionic InRelease
Hit:4 http://pk.archive.ubuntu.com/ubuntu bionic InRelease
Hit:5 http://dl.google.com/linux/chrome/deb stable Release
Hit:6 http://pk.archive.ubuntu.com/ubuntu bionic-updates InRelease
Hit:8 http://pk.archive.ubuntu.com/ubuntu bionic-backports InRelease
Reading package lists... Done
```

```
tariqjc@tariqjc:~$ sudo apt-get install \
> apt-transport-https \
> ca-certificates \
> curl \
> gnupg-agent \
> software-properties-common
```

```
tariqjc@tariqjc:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
OK
tariqjc@tariqjc:~$ sudo apt-key fingerprint 0EBFCD88
pub  rsa4096 2017-02-22 [SCEA]
     9DC8 5822 9FC7 DD38 854A  E2D8 8D81 803C 0EBF CD88
uid  [ unknown] Docker Release (CE deb) <docker@docker.com>
sub  rsa4096 2017-02-22 [S]
```

Now, Install using the convenience script:

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
```

```
tariqjc@tariqjc: ~  
File Edit View Search Terminal Help  
tariqjc@tariqjc:~$ sudo curl -fsSL https://get.docker.com -o get-docker.sh
```

```
tariqjc@tariqjc: ~  
File Edit View Search Terminal Help  
tariqjc@tariqjc:~$ sudo curl -fsSL https://get.docker.com -o get-docker.sh  
tariqjc@tariqjc:~$ sudo sh get-docker.sh  
# Executing docker install script, commit: 1b02882d63b9cfc484ad6b0180171c679cfe0  
f3a  
+ sh -c apt-get update -qq >/dev/null
```

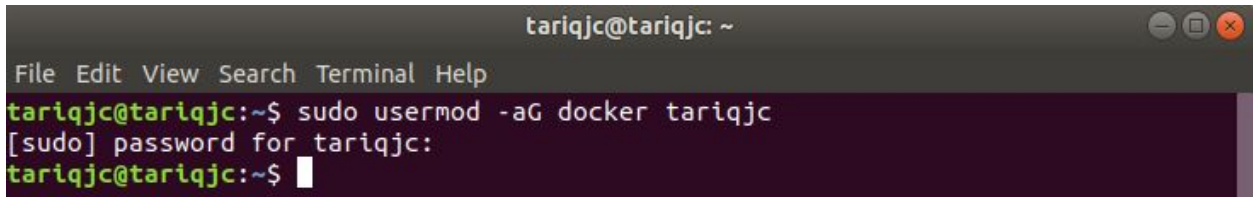
The following information is displayed:

```
Client: Docker Engine - Community  
Version: 19.03.8  
API version: 1.40  
Go version: go1.12.17  
Git commit: afacb8b7f0  
Built: Wed Mar 11 01:25:46 2020  
OS/Arch: linux/amd64  
Experimental: false  
  
Server: Docker Engine - Community  
Engine:  
Version: 19.03.8  
API version: 1.40 (minimum version 1.12)  
Go version: go1.12.17  
Git commit: afacb8b7f0  
Built: Wed Mar 11 01:24:19 2020  
OS/Arch: linux/amd64  
Experimental: false  
containerd:  
Version: 1.2.13  
GitCommit: 7ad184331fa3e55e52b890ea95e65ba581ae3429  
runc:  
Version: 1.0.0-rc10  
GitCommit: dc9208a3303feef5b3839f4323d9beb36df0a9dd  
docker-init:  
Version: 0.18.0  
GitCommit: fec3683  
  
If you would like to use Docker as a non-root user, you should now consider  
adding your user to the "docker" group with something like:  
  
    sudo usermod -aG docker your-user  
  
Remember that you will have to log out and back in for this to take effect!  
  
WARNING: Adding a user to the "docker" group will grant the ability to run  
containers which can be used to obtain root privileges on the  
docker host.  
Refer to https://docs.docker.com/engine/security/security/#docker-daemon-attack-surface  
for more information.
```

If you would like to use Docker as a non-root user, you should now consider adding your user to the “docker” group with something like:

`sudo usermod -aG docker your-user`

After which you should log out and login for it to take effect

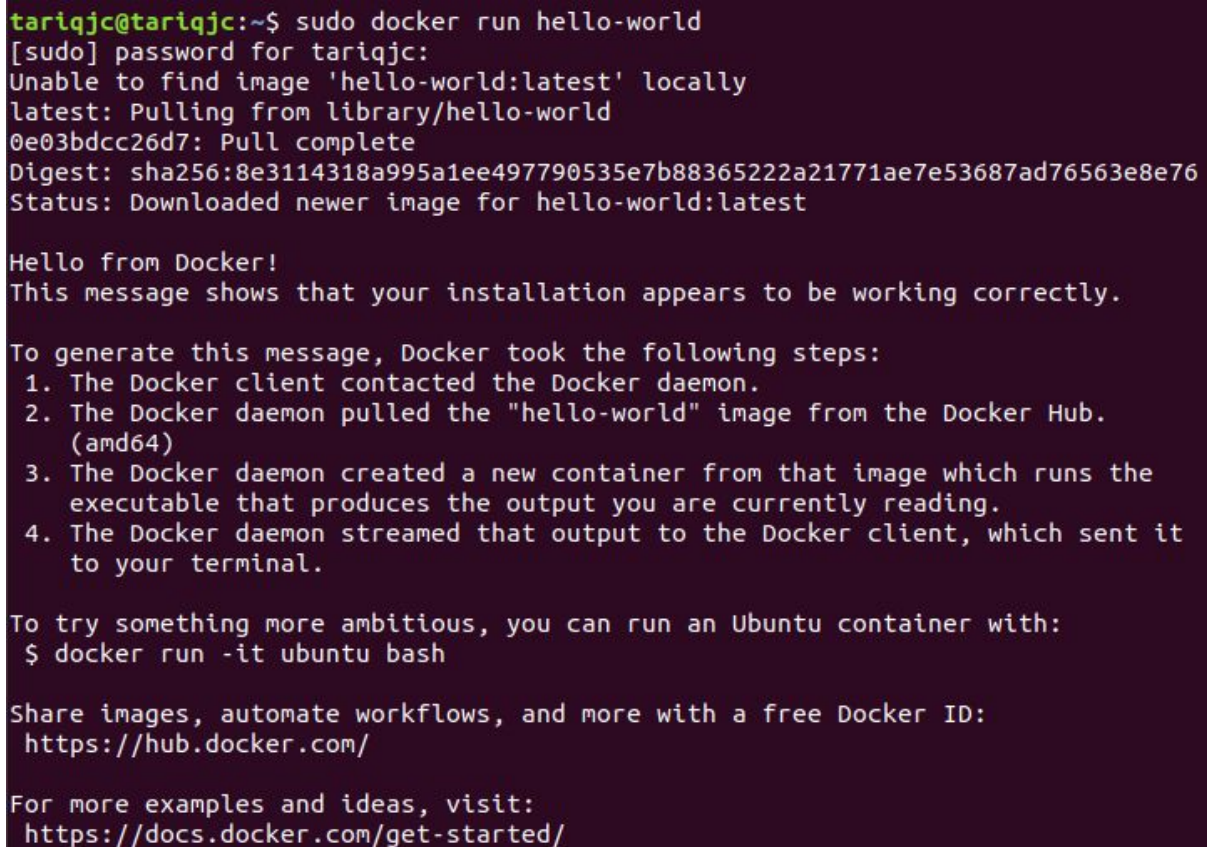
A terminal window titled 'tariqjc@tariqjc: ~' with a menu bar (File, Edit, View, Search, Terminal, Help). The terminal shows the command 'sudo usermod -aG docker tariqjc' being entered. It prompts for a password, which is entered, and then shows the command prompt again.

```
tariqjc@tariqjc: ~  
File Edit View Search Terminal Help  
tariqjc@tariqjc:~$ sudo usermod -aG docker tariqjc  
[sudo] password for tariqjc:  
tariqjc@tariqjc:~$
```

Docker Commands

`sudo docker run hello-world`

The above will pull the image and execute it and print “Hello from Docker”

A terminal window showing the output of the command 'sudo docker run hello-world'. It prompts for a password, then shows the process of pulling the 'hello-world:latest' image from Docker Hub. The output includes the image digest and a confirmation that the image was downloaded. Below this, it says 'Hello from Docker!' and provides a message about the installation. It then lists the steps Docker took to generate the message. Finally, it suggests running an Ubuntu container and provides links to Docker Hub and documentation.

```
tariqjc@tariqjc:~$ sudo docker run hello-world  
[sudo] password for tariqjc:  
Unable to find image 'hello-world:latest' locally  
latest: Pulling from library/hello-world  
0e03bdcc26d7: Pull complete  
Digest: sha256:8e3114318a995a1ee497790535e7b88365222a21771ae7e53687ad76563e8e76  
Status: Downloaded newer image for hello-world:latest  
  
Hello from Docker!  
This message shows that your installation appears to be working correctly.  
  
To generate this message, Docker took the following steps:  
1. The Docker client contacted the Docker daemon.  
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.  
   (amd64)  
3. The Docker daemon created a new container from that image which runs the  
   executable that produces the output you are currently reading.  
4. The Docker daemon streamed that output to the Docker client, which sent it  
   to your terminal.  
  
To try something more ambitious, you can run an Ubuntu container with:  
$ docker run -it ubuntu bash  
  
Share images, automate workflows, and more with a free Docker ID:  
https://hub.docker.com/  
  
For more examples and ideas, visit:  
https://docs.docker.com/get-started/
```

Register yourself at docker hub:

- go to hub.docker.com (create dockerID)

- ```
sudo docker run docker/whalesay cowsay boo
```

```
Digest: sha256:178598e51a26abbc958b8a2e48825c90bc22e641de3d31e18aaf55f3258ba93b
Status: Downloaded newer image for docker/whalesay:latest
```

```
< boo >

 \ /
 \ /
 \ /
 \/#####.
 ## ## ## ==
 ## ## ## ===
  ~~~ {~~~~ o ~~~~} ===== ~~~~  
       |_____|  
       |_____|  
       |_____|  
       |_____|  
tarigjc@tarigjc:~$
```

Let's create two more containers for the same image:

```
tariqjc@tariqjc:~$ sudo docker run docker/whalesay cowsay tweedledum
```

< tweedledum >

```
tariqjc@tariqjc:~$ sudo docker run docker/whalesay cowsay tweedledee
```

< tweedledee >

Try running the nginx docker container:

```
docker run nginx
```

(I ran this several times; the first time it downloaded the image and ran it and exited)

```
docker ps
```

List all running containers (IDs etc.)

```
tariqjc@tariqjc:~$ docker ps
```

| CONTAINER ID | IMAGE | COMMAND | CREATED |
|--------------|-------|---------|---------|
| STATUS       | PORTS | NAMES   |         |

As you can see, none of the containers are running at the moment. All stopped after running. I forced my nginx containers to stop through CTRL+C

Every container has an ID (given by docker), its image name, the command used to run it, time it was created, its current status (running or exited), assigned ports (if any by the user) and a name (given by docker)

## docker ps -a

Show all prev and current containers

```
tariqjc@tariqjc:~$ docker ps -a
```

| CONTAINER ID              | IMAGE           | COMMAND                  | CREATED        |
|---------------------------|-----------------|--------------------------|----------------|
| STATUS                    | PORTS           | NAMES                    |                |
| 4a673f7a8c82              | nginx           | "nginx -g 'daemon of..." | 18 minutes ago |
| Exited (0) 4 minutes ago  |                 | objective_kalam          |                |
| be22b01c6614              | nginx           | "nginx -g 'daemon of..." | 19 minutes ago |
| Exited (0) 18 minutes ago |                 | youthful_hellman         |                |
| 03bcebad799c              | nginx           | "nginx -g 'daemon of..." | 8 hours ago    |
| Exited (0) 19 minutes ago |                 | quizzical_maxwell        |                |
| 158b285dd964              | docker/whalesay | "cowsay tweedledee"      | 8 hours ago    |
| Exited (0) 8 hours ago    |                 | charming_lehmann         |                |
| 32cd27d13614              | docker/whalesay | "cowsay tweedledum"      | 8 hours ago    |
| Exited (0) 8 hours ago    |                 | gallant_feistel          |                |
| 738c3962d107              | docker/whalesay | "cowsay boo"             | 8 hours ago    |
| Exited (0) 8 hours ago    |                 | trusting_davinci         |                |
| 5ca69da7fd8e              | hello-world     | "/hello"                 | 8 hours ago    |
| Exited (0) 8 hours ago    |                 | serene_brahmagupta       |                |

Now, I ran docker run nginx in a separate terminal and checked the running containers:

```
tariqjc@tariqjc:~$ docker ps
```

| CONTAINER ID | IMAGE  | COMMAND                  | CREATED        |
|--------------|--------|--------------------------|----------------|
| STATUS       | PORTS  | NAMES                    |                |
| f9834aec39ed | nginx  | "nginx -g 'daemon of..." | 11 seconds ago |
| Up 9 seconds | 80/tcp | affectionate_zhukovsky   |                |

```
tariqjc@tariqjc:~$
```

## docker stop ID/NAME

I can stop a running docker with either its name or ID. A message will confirm.

```
tariqjc@tariqjc:~$ docker stop affectionate_zhukovsky
affectionate_zhukovsky
tariqjc@tariqjc:~$
```

## docker rm name/id

get rid container permanently:

After stopping, the docker will still remain in the history list....

```
tariqjc@tariqjc:~$ docker ps -a
```

| CONTAINER ID              | IMAGE | COMMAND                  | CREATED       |
|---------------------------|-------|--------------------------|---------------|
| STATUS                    | PORTS | NAMES                    |               |
| f9834aec39ed              | nginx | "nginx -g 'daemon of..." | 3 minutes ago |
| Exited (0) 55 seconds ago |       | affectionate_zhukovsky   |               |

Remove it completely through the name:

```
tariqjc@tariqjc:~$ docker rm affectionate_zhukovsky
affectionate_zhukovsky
tariqjc@tariqjc:~$ docker ps -a
```

| CONTAINER ID              | IMAGE           | PORTS | COMMAND                  | NAMES              | CREATED        |
|---------------------------|-----------------|-------|--------------------------|--------------------|----------------|
| 4a673f7a8c82              | nginx           |       | "nginx -g 'daemon of..." | objective_kalam    | 25 minutes ago |
| Exited (0) 10 minutes ago |                 |       |                          |                    |                |
| be22b01c6614              | nginx           |       | "nginx -g 'daemon of..." | youthful_hellman   | 26 minutes ago |
| Exited (0) 25 minutes ago |                 |       |                          |                    |                |
| 03bcebad799c              | nginx           |       | "nginx -g 'daemon of..." | quizzical_maxwell  | 8 hours ago    |
| Exited (0) 26 minutes ago |                 |       |                          |                    |                |
| 158b285dd964              | docker/whalesay |       | "cowsay tweedledee"      | charming_lehmann   | 8 hours ago    |
| Exited (0) 8 hours ago    |                 |       |                          |                    |                |
| 32cd27d13614              | docker/whalesay |       | "cowsay tweedledum"      | gallant_feistel    | 8 hours ago    |
| Exited (0) 8 hours ago    |                 |       |                          |                    |                |
| 738c3962d107              | docker/whalesay |       | "cowsay boo"             | trusting_davinci   | 8 hours ago    |
| Exited (0) 8 hours ago    |                 |       |                          |                    |                |
| 5ca69da7fd8e              | hello-world     |       | "/hello"                 | serene_brahmagupta | 8 hours ago    |
| Exited (0) 8 hours ago    |                 |       |                          |                    |                |

## docker images

List all the docker images downloaded from registry

```
tariqjc@tariqjc:~$ docker images
```

| REPOSITORY      | TAG    | IMAGE ID     | CREATED      | SIZE  |
|-----------------|--------|--------------|--------------|-------|
| nginx           | latest | 602e111c06b6 | 2 weeks ago  | 127MB |
| hello-world     | latest | bf756fb1ae65 | 4 months ago | 13.3k |
| B               |        |              |              |       |
| docker/whalesay | latest | 6b362a9f73eb | 4 years ago  | 247MB |

## docker rmi

### docker rmi nginx

Removes an image but we have to ensure that all of its containers are deleted before:

Let us delete all nginx containers

```
tariqjc@tariqjc:~$ docker ps -a
```

| CONTAINER ID              | IMAGE           | COMMAND                   | CREATED        |
|---------------------------|-----------------|---------------------------|----------------|
| 4a673f7a8c82              | nginx           | "nginx -g 'daemon of...'" | 26 minutes ago |
| Exited (0) 12 minutes ago |                 | objective_kalam           |                |
| be22b01c6614              | nginx           | "nginx -g 'daemon of...'" | 27 minutes ago |
| Exited (0) 27 minutes ago |                 | youthful_hellman          |                |
| 03bcebad799c              | nginx           | "nginx -g 'daemon of...'" | 8 hours ago    |
| Exited (0) 27 minutes ago |                 | quizzical_maxwell         |                |
| 158b285dd964              | docker/whalesay | "cowsay tweedledee"       | 8 hours ago    |
| Exited (0) 8 hours ago    |                 | charming_lehmann          |                |
| 32cd27d13614              | docker/whalesay | "cowsay tweedledum"       | 8 hours ago    |
| Exited (0) 8 hours ago    |                 | gallant_feistel           |                |
| 738c3962d107              | docker/whalesay | "cowsay boo"              | 8 hours ago    |
| Exited (0) 8 hours ago    |                 | trusting_davinci          |                |
| 5ca69da7fd8e              | hello-world     | "/hello"                  | 8 hours ago    |
| Exited (0) 8 hours ago    |                 | serene_brahmagupta        |                |

```
tariqjc@tariqjc:~$ docker rm objective_kalam
objective_kalam
tariqjc@tariqjc:~$ docker rm youthful_hellman
youthful_hellman
tariqjc@tariqjc:~$ docker rm quizzical_maxwell
quizzical_maxwell
```

Now, I can delete the nginx image

```
tariqjc@tariqjc:~$ docker rmi nginx
Untagged: nginx:latest
Untagged: nginx@sha256:86ae264c3f4acb99b2dee4d0098c40cb8c46dcf9e1148f05d3a51c4df6758c12
Deleted: sha256:602e111c06b6934013578ad80554a074049c59441d9bcd963cb4a7feccede7a5
Deleted: sha256:81eaddad75aaa517b4a597912da28c2f5b905f6e9789dce3aea874b040aad201
Deleted: sha256:73cafa8418003ecfaa02360f181c132b2cf4b61433e1bd5c84012941105865c8
Deleted: sha256:c2adabaecedbda0af72b153c6499a0555f3a769d52370469d8f6bd6328af9b13
tariqjc@tariqjc:~$ docker images
```

| REPOSITORY      | TAG    | IMAGE ID     | CREATED      | SIZE  |
|-----------------|--------|--------------|--------------|-------|
| hello-world     | latest | bf756fb1ae65 | 4 months ago | 13.3k |
| B               |        |              |              |       |
| docker/whalesay | latest | 6b362a9f73eb | 4 years ago  | 247MB |

```
tariqjc@tariqjc:~$
```

## docker pull nginx

The above will only pull image but not run it

```
tariqjc@tariqjc:~$ docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
54fec2fa59d0: Pull complete
4ede6f09aefe: Pull complete
f9dc69acb465: Pull complete
Digest: sha256:86ae264c3f4acb99b2dee4d0098c40cb8c46dcf9e1148f05d3a51c4df6758c12
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

Verify in image list

```
tariqjc@tariqjc:~$ docker images
```

| REPOSITORY      | TAG    | IMAGE ID     | CREATED      | SIZE  |
|-----------------|--------|--------------|--------------|-------|
| nginx           | latest | 602e111c06b6 | 2 weeks ago  | 127MB |
| hello-world     | latest | bf756fb1ae65 | 4 months ago | 13.3k |
| B               |        |              |              |       |
| docker/whalesay | latest | 6b362a9f73eb | 4 years ago  | 247MB |

Check to see no running or previous containers of nginx

```
tariqjc@tariqjc:~$ docker ps
```

| CONTAINER ID | IMAGE | COMMAND | CREATED | STATUS |
|--------------|-------|---------|---------|--------|
|              |       |         |         |        |

```
tariqjc@tariqjc:~$ docker ps -a
```

| CONTAINER ID        | IMAGE           | COMMAND             | CREATED     | STATUS |
|---------------------|-----------------|---------------------|-------------|--------|
| 158b285dd964        | docker/whalesay | "cowsay tweedledee" | 8 hours ago | Exited |
| ted (0) 8 hours ago |                 | charming_lehmann    |             |        |
| 32cd27d13614        | docker/whalesay | "cowsay tweedledum" | 8 hours ago | Exited |
| ted (0) 8 hours ago |                 | gallant_feistel     |             |        |
| 738c3962d107        | docker/whalesay | "cowsay boo"        | 8 hours ago | Exited |
| ted (0) 8 hours ago |                 | trusting_davinci    |             |        |
| 5ca69da7fd8e        | hello-world     | "/hello"            | 8 hours ago | Exited |
| ted (0) 8 hours ago |                 | serene_brahmagupta  |             |        |

I can use the -it command to work from inside a container

Create a whalesay container and echo some commands:

**docker run -it docker/whalesay sh**

```
tariqjc@tariqjc:~$ docker run -it docker/whalesay sh
```

```
# echo hello
hello
# ls
ChangeLog  LICENSE  README  cows  cowsay.1  install.sh
INSTALL    MANIFEST  Wrap.pm.diff  cowsay  install.pl  pgp_public_key.txt
# mkdir bdaclass
# ls
ChangeLog  MANIFEST  bdaclass  cowsay.1  pgp_public_key.txt
INSTALL    README    cows      install.pl
LICENSE    Wrap.pm.diff  cowsay    install.sh
# exit
```

You can remove the sensitive system files within the container and exit. It will make a difference only to this container. You can simply delete this container and start a new one

```
tariqjc@tariqjc:~$ docker run -it docker/whalesay sh
# ls
ChangeLog  LICENSE  README      cows    cowsay.1  install.sh
INSTALL    MANIFEST  Wrap.pm.diff cowsay   install.pl pgp_public_key.txt
# rm install.sh
# rm cowsay
# rm pgp_public_key.txt
# ls
ChangeLog  LICENSE  README      cows    install.pl
INSTALL    MANIFEST  Wrap.pm.diff cowsay.1
# exit
```

Lets deploy a web application with docker:

**docker run --rm prakhar1989/static-site**

```
tariqjc@tariqjc:~$ docker run --rm prakhar1989/static-site
Unable to find image 'prakhar1989/static-site:latest' locally
latest: Pulling from prakhar1989/static-site
d4bce7fd68df: Pull complete
a3ed95caeb02: Pull complete
573113c4751a: Pull complete
31917632be33: Pull complete
77e66f18af1c: Pull complete
df3f108f3ade: Pull complete
d7a279eb19f5: Pull complete
e798589c05c5: Pull complete
78eeaf458ae0: Pull complete
Digest: sha256:bb6907c8db9ac4c6cadb25162a979e286575cd8b27727c08c7fbaf30988534db
Status: Downloaded newer image for prakhar1989/static-site:latest
Nginx is running...
```

The --rm flag will automatically remove the container when it exits. Right now, the container is up and running but we need to specify some more commands to see the content on website (e.g., ports)

Stop the container (you can see it also deleted itself):

```

Ctariqjc@tariqjc:~$ docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STA
TUS                PORTS              NAMES
7d2269403d99       docker/whalesay    "sh"               17 minutes ago     Exi
ted (0) 17 minutes ago      stoic_rhodes
c61ed4d20fe3       docker/whalesay    "sh"               19 minutes ago     Exi
ted (0) 18 minutes ago      agitated_zhukovsky
b330bc60610a       docker/whalesay    "sh"               21 minutes ago     Exi
ted (0) 20 minutes ago      wizardly_rhodes
158b285dd964       docker/whalesay    "cowsay tweedledee" 8 hours ago        Exi
ted (0) 8 hours ago        charming_lehmann
82cd27d13614       docker/whalesay    "cowsay tweedledum" 8 hours ago        Exi
ted (0) 8 hours ago        gallant_feistel
738c3962d107       docker/whalesay    "cowsay boo"        8 hours ago        Exi
ted (0) 8 hours ago        trusting_davinci
5ca69da7fd8e       hello-world        "/hello"           9 hours ago        Exi
ted (0) 9 hours ago        serene_brahmagupta

```

Now, type as follows:

**docker run -d -P --name static-site prakhar1989/static-site**

```

tariqjc@tariqjc:~$ docker run -d -P --name static-site prakhar1989/static-site
7ff7771c205e55d5c7ec5a18bf65570866e6efdb0dc16de9d178ac14d7b90761

```

Here, -d will make the website run in background (so I can continue to type commands at the terminal), -P will assign random ports, and --name will assign a name as we want. The output confirms the execution

Now, check the ports:

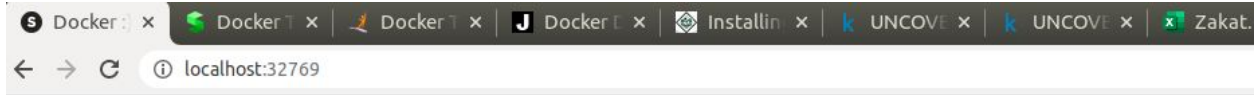
**docker port static-site**

```

tariqjc@tariqjc:~$ docker port static-site
443/tcp -> 0.0.0.0:32768
80/tcp -> 0.0.0.0:32769
tariqjc@tariqjc:~$

```

Now, check on browser (<http://localhost:32769/>):



Hello Docker!

This is being served from a **docker** container running Nginx.

Lets say I want to specify my own port. First stop the previous container and then specify your own:

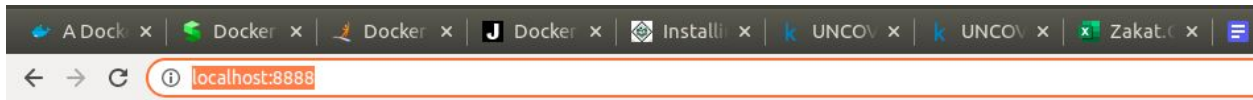
```
tariqjc@tariqjc:~$ docker stop 7ff77  
7ff77
```

**docker run -d -p 8888:80 prakkar1989/static-site**

```
tariqjc@tariqjc:~$ docker run -d -p 8888:80 prakkar1989/static-site  
7542ccbdc57a9efa71b103d08bb7cf5acdd014ee2205f394d02926d69ef2f976
```

In this case, I want to map the network service port 80 of the container to port 8888 on my host machine, i.e., whatever should be visible on port 80 inside the container will be visible on port 8888 on my machine

Cross-check:



Now, stop the containers

## Dockerfile example

Create the following Dockerfile with nano (nano Dockerfile) and add the following:

Add the following:

#This is a sample Image

FROM ubuntu

MAINTAINER bdaclass2020

RUN apt-get update

CMD ["echo","Image created"]

Now, save and exit and type

**docker build -t demoimage:0.1 .**

```

tariqjc@tariqjc:~$ docker build -t demoimage:0.1 .
Sending build context to Docker daemon 4.607GB
Step 1/4 : FROM ubuntu
----> 1d622ef86b13
Step 2/4 : MAINTAINER bdaclass2020
----> Running in 3831cb698fbb
Removing intermediate container 3831cb698fbb
----> b8722139e78f
Step 3/4 : RUN apt-get update
----> Running in 6b8e034d7966
Get:1 http://archive.ubuntu.com/ubuntu focal InRelease [265 kB]
Get:2 http://security.ubuntu.com/ubuntu focal-security InRelease [107 kB]
Get:3 http://security.ubuntu.com/ubuntu focal-security/restricted amd64 Packages [4673 B]
Get:4 http://archive.ubuntu.com/ubuntu focal-updates InRelease [107 kB]
Get:5 http://security.ubuntu.com/ubuntu focal-security/universe amd64 Packages [8273 B]
Get:6 http://security.ubuntu.com/ubuntu focal-security/main amd64 Packages [60.9 kB]
Get:7 http://archive.ubuntu.com/ubuntu focal-backports InRelease [98.3 kB]
Get:8 http://archive.ubuntu.com/ubuntu focal/restricted amd64 Packages [33.4 kB]
Get:9 http://archive.ubuntu.com/ubuntu focal/universe amd64 Packages [11.3 MB]
Get:10 http://archive.ubuntu.com/ubuntu focal/main amd64 Packages [1275 kB]
Get:11 http://archive.ubuntu.com/ubuntu focal/multiverse amd64 Packages [177 kB]
Get:12 http://archive.ubuntu.com/ubuntu focal-updates/restricted amd64 Packages [4673 B]
Get:13 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 Packages [105 kB]
Get:14 http://archive.ubuntu.com/ubuntu focal-updates/universe amd64 Packages [32.4 kB]
Get:15 http://archive.ubuntu.com/ubuntu focal-backports/universe amd64 Packages [2903 B]
Fetched 13.6 MB in 30s (455 kB/s)
Reading package lists...
Removing intermediate container 6b8e034d7966
----> ef2d7778e46e
Step 4/4 : CMD ["echo","Image created"]
----> Running in dc522210b117
Removing intermediate container dc522210b117
----> 3b5be62b5959
Successfully built 3b5be62b5959
Successfully tagged demoimage:0.1

```

-t is for tag (0.1) and the . after the space specifies the current directory in which the Dockerfile is present

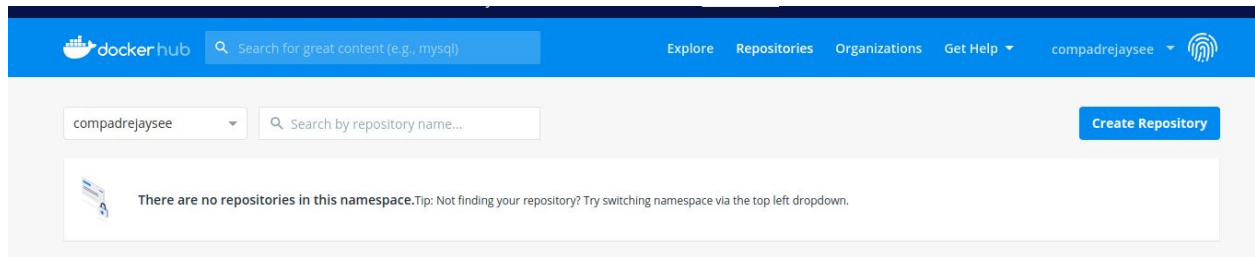
Verify in list of images:

```

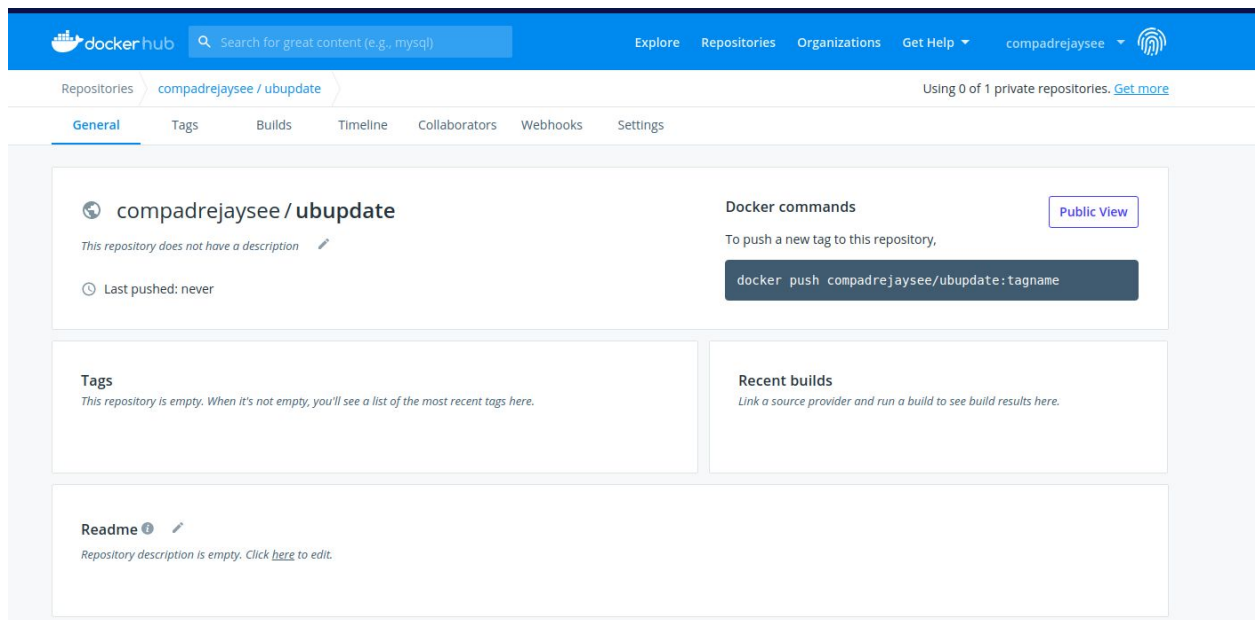
tariqjc@tariqjc:~$ docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
demoimage            0.1                3b5be62b5959       3 minutes ago      95.3MB

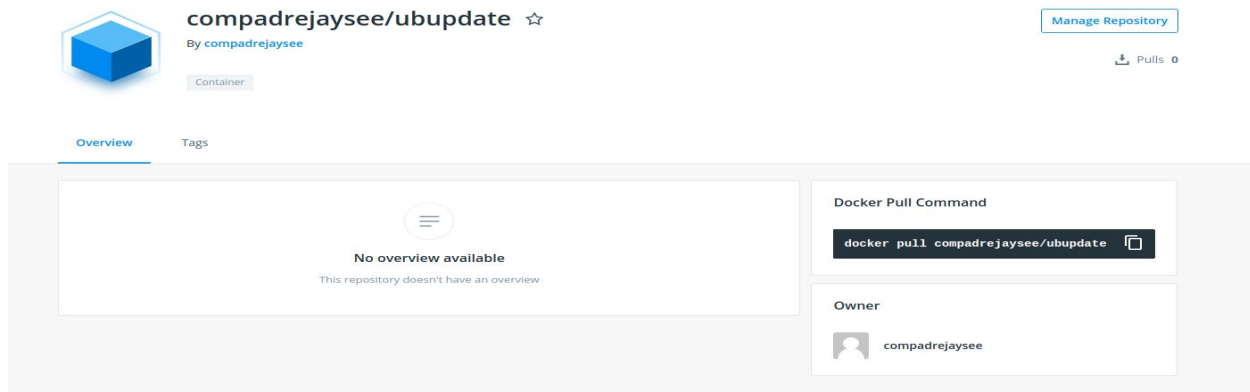
```

You will now need to setup a docker hub repo to upload this image for usage.



Create a repo called ubupdate in public domain





Now, open terminal and login to docker:

```
tariqjc@tariqjc: ~  
File Edit View Search Terminal Help  
tariqjc@tariqjc:~$ docker login  
Login with your Docker ID to push and pull images from Docker Hub. If you don't  
have a Docker ID, head over to https://hub.docker.com to create one.  
Username: compadrejaysee  
Password:  
WARNING! Your password will be stored unencrypted in /home/tariqjc/.docker/conn  
fig.json.  
Configure a credential helper to remove this warning. See  
https://docs.docker.com/engine/reference/commandline/login/#credentials-store  
  
Login Succeeded  
tariqjc@tariqjc:~$
```

Tag and image with repository

Now, tag the image ID to your repository

**docker tag 3b5be62b5959 compadrejaysee/ubupdate:1.0**

```
tariqjc@tariqjc:~$ docker tag 3b5be62b5959 compadrejaysee/ubupdate:1.0
```

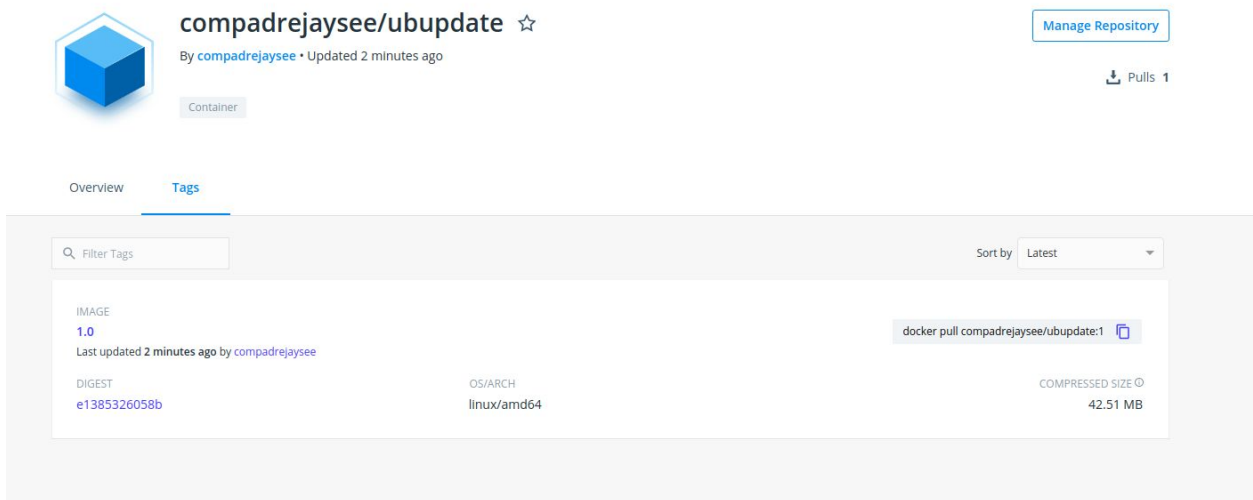
In above, the tag is the same as the tag of our image

Now, push the image to the repository using the command:

**docker push compadrejaysee/ubupdate:1.0**

```
tariqjc@tariqjc:~$ docker push compadrejaysee/ubupdate:1.0
The push refers to repository [docker.io/compadrejaysee/ubupdate]
bfc0fa7342aa: Pushed
8891751e0a17: Pushed
2a19bd70fcd4: Pushed
9e53fd489559: Pushed
7789f1a3d4e9: Pushed
1.0: digest: sha256:e1385326058ba228881b4431397b37c7867b214257857dda8066bf176bdbfe04 size: 1364
tariqjc@tariqjc:~$
```

Verify on the website



The screenshot shows the Docker Hub interface for the repository **compadrejaysee/ubupdate**. The repository is marked as a "Container" and was updated 2 minutes ago. It has 1 pull. The "Tags" tab is selected, showing a list of tags. The tag **1.0** is highlighted, with a last update time of 2 minutes ago. The digest is **e1385326058b**, the OS/ARCH is **linux/amd64**, and the compressed size is **42.51 MB**. A search bar for tags and a sort dropdown set to "Latest" are also visible.

| IMAGE                                                      | DIGEST              | OS/ARCH            | COMPRESSED SIZE |
|------------------------------------------------------------|---------------------|--------------------|-----------------|
| <b>1.0</b><br>Last updated 2 minutes ago by compadrejaysee | <b>e1385326058b</b> | <b>linux/amd64</b> | <b>42.51 MB</b> |

The pull command is as follows:

**docker pull compadrejaysee/ubupdate**

```
tariqjc@tariqjc:~$ docker pull compadrejaysee/ubupdate:1.0
1.0: Pulling from compadrejaysee/ubupdate
Digest: sha256:e1385326058ba228881b4431397b37c7867b214257857dda8066bf176bdbfe04
Status: Image is up to date for compadrejaysee/ubupdate:1.0
docker.io/compadrejaysee/ubupdate:1.0
tariqjc@tariqjc:~$ docker images
```

| REPOSITORY              | TAG    | IMAGE ID     | CREATED        | SIZE   |
|-------------------------|--------|--------------|----------------|--------|
| compadrejaysee/ubupdate | 1.0    | 3b5be62b5959 | 27 minutes ago | 95.3MB |
| demoimage               | 0.1    | 3b5be62b5959 | 27 minutes ago | 95.3MB |
| ubupdate                | latest | 3b5be62b5959 | 27 minutes ago | 95.3MB |
| ubuntu                  | latest | 1d622ef86b13 | 2 weeks ago    | 73.9MB |
| nginx                   | latest | 602e111c06b6 | 2 weeks ago    | 127MB  |
| hello-world             | latest | bf756fb1ae65 | 4 months ago   | 13.3kB |
| prakhar1989/static-site | latest | f01030e1dcf3 | 4 years ago    | 134MB  |
| docker/whalesay         | latest | 6b362a9f73eb | 4 years ago    | 247MB  |

```
tariqjc@tariqjc:~$
```

## Docker Exercises:

For more help, see the following URLs:

- <http://tutorials.jenkov.com/docker/dockerfile.html>
- <https://intellipaat.com/blog/tutorial/devops-tutorial/docker-tutorial/>
- <https://stackify.com/docker-tutorial/>

### Exercise 1:

Start 3 containers from image that does not automatically exit, such as nginx, detached.  
 Stop 2 of the containers leaving 1 up.  
 Submitting the output for docker ps -a is enough to prove this exercise has been done.

### Exercise 2:

Clean the docker daemon from all images and containers.  
 Submit the output for docker ps -a and docker images

### Exercise 3:

Start image devopsdockeruh/pull\_exercise with flags -it like so: docker run -it devopsdockeruh/pull\_exercise. It will wait for your input. Navigate through docker hub to find the docs and Dockerfile that was used to create the image.  
 Read the Dockerfile and/or docs to learn what input will get the application to answer a "secret message".  
 Submit the secret message and command(s) given to get it as your answer.

#### Exercise 4:

Start image devopsdockeruh/exec\_bash\_exercise, it will start a container with clock-like features and create a log. Go inside the container and use `tail -f ./logs.txt` to follow the logs. Every 15 seconds the clock will send you a “secret message”. Submit the secret message and command(s) given as your answer.

#### Exercise 5:

Start a ubuntu image with the process `sh -c 'echo "Input website:"; read website; echo "Searching.."; sleep 1; curl http://$website;'`

You will notice that a few things required for proper execution are missing. Be sure to remind yourself which flags to use so that the read actually waits for input.

Note also that curl is NOT installed in the container yet. You will have to install it from inside of the container.

Test inputting `helsinki.fi` into the application. It should respond with something like

```
<html>
<head>
  <title>301 Moved Permanently</title>
</head>
<body>
  <h1>Moved Permanently</h1>
  <p>The document has moved <a href="http://www.helsinki.fi/">here</a>.</p>
</body>
</html>
```

This time return the command you used to start the process and the command(s) you used to fix the ensuing problems.

*This exercise has multiple solutions, if the curl for helsinki.fi works then it's done. Can you figure out other (smart) solutions?*

### Exercise 6:

Create a Dockerfile that starts with `FROM devopsdockeruh/overwrite_cmd_exercise` and works only as a clock.

The developer has poorly documented how the application works. Passing flags will open different functionalities, but we'd like to create a simplified version of it.

Add a CMD line to the Dockerfile and tag it as "docker-clock" so that `docker run docker-clock` starts the application and the clock output.

Return both Dockerfile(s) and the command you used to run the container(s)

### Exercise 7:

Make a script file for `echo "Input website:"; read website; echo "Searching.."; sleep 1; curl http://$website; and run it inside the container using CMD. Build the image with tag "curler".`

Run command `docker run [options] curler` (with correct flags again, as in 1.5) and input `helsinki.fi` into it. Output should match the 1.5 one.

Return both Dockerfile(s) and the command you used to run the container(s)

### Exercise 8:

In this exercise we won't create a new Dockerfile. Image `devopsdockeruh/ports_exercise` will start a web service in port 80. Use -p flag to access the contents with your browser.

Submit your used commands for this exercise.

### Exercise 9:

Create Dockerfile for an application in any of your own repositories and publish it to Docker Hub. This can be any project except clones / forks of backend-example or frontend-example.

For this exercise to be complete you have to provide the link to the project in docker hub, make sure you at least have a basic description and instructions for how to run the application in a [README](#) that's available through your submission.

#### Exercise 10:

Create an image that contains your favorite programming environment in it's entirety.

This means that a computer that only has docker can use the image to start a container which contains all the tools and libraries. Excluding IDE / Editor. The environment can be partially used by running commands manually inside the container.

Explain what you created and publish it to Docker Hub.